

Pandas – Complete Guide with Examples and Explanations

What is Pandas?

Pandas is a Python library designed for data analysis and manipulation. It allows you to work with structured data in the form of tables.

It supports tasks such as:

- Loading and reading data from files
- Cleaning and transforming data
- Aggregating and analyzing
- Visualizing and exporting data

It is built on top of NumPy and integrates well with Matplotlib, Seaborn, and Scikit-learn.

Core Data Structures in Pandas

1. Series (1D)

A Series is like a column in Excel: it's a one-dimensional labeled array.

```
import pandas as pd

data = pd.Series([10, 20, 30, 40])
print(data)
```

Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

Each element has a label called an index. You can specify custom indexes:

```
data = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(data['b']) # Output: 20
```

2. DataFrame (2D)

A DataFrame is like a full spreadsheet with rows and columns.

```
data = {
    'Name': ['Ali', 'Zara', 'Ahmed'],
    'Age': [25, 30, 22],
    'City': ['Lahore', 'Karachi', 'Islamabad']
}

df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Age	City
0	Ali	25	Lahore
1	Zara	30	Karachi
2	Ahmed	22	Islamabad

Reading Data from Files

```
df = pd.read_csv('data.csv')           # Reads data from a CSV file
df = pd.read_excel('file.xlsx')         # Reads data from Excel
df = pd.read_json('data.json')          # Reads data from JSON
```

Writing Data to Files

```
df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', index=False)
```

Inspecting the Data

These methods help you understand the contents of a DataFrame:

```
df.head()           # First 5 rows
df.tail()           # Last 5 rows
df.shape            # Returns (rows, columns)
df.info()           # Data types and non-null counts
df.describe()       # Summary statistics for numeric columns
```

Selecting and Filtering Data

```
df['Age']            # Select one column
df[['Name', 'City']] # Select multiple columns
df.iloc[1]          # Row by index (position)
df.loc[0]            # Row by label (index value)
df[df['Age'] > 25]    # Filter rows with condition
```

Modifying Data

```
df['Salary'] = [50000, 60000, 45000] # Add new column
df.loc[0, 'Age'] = 26                  # Update specific value
df['Age_in_5_years'] = df['Age'].apply(lambda x: x + 5) # Apply function to column
```

Handling Missing Data

Missing values (NaNs) often occur. Here's how to handle them:

```
df.isnull()          # Detect missing values
df.fillna(0)          # Replace missing with 0
df.fillna(df['Age'].mean()) # Replace with column mean
df.dropna()           # Remove rows with any missing data
```

Replacing Values

```
df['City'] = df['City'].replace({'Lahore': 'LHE', 'Karachi': 'KHI'})
```

This replaces specific values in the column.

Combining DataFrames

Merging (like SQL joins)

```
pd.merge(df1, df2, on='ID', how='inner') # other options: 'left', 'right', 'outer'
```

Concatenation (stacking)

```
pd.concat([df1, df2]) # Adds rows from df2 under df1
```

Grouping and Aggregation

```
df.groupby('City')['Salary'].mean() # Mean salary per city
df.groupby('City').size()           # Number of entries per city
```

Useful for summarizing grouped data.

Pivot Tables

```
pd.pivot_table(df, values='Salary', index='City', columns='Gender', aggfunc='mean')
```

Useful when you want to cross-tabulate data with multiple categories.

Reshaping Data: Melt and Pivot

```
pd.melt(df, id_vars=['Name'], value_vars=['Age', 'Salary']) # Unpivot
df.pivot(index='Name', columns='Attribute', values='Value') # Re-pivot
```

Cleaning Text Data

```
df['Name'] = df['Name'].str.strip() # Remove spaces
df['City'] = df['City'].str.lower() # Convert to lowercase
df.duplicated()                    # Find duplicates
df.drop_duplicates()               # Remove duplicates
df['Age'] = df['Age'].astype(int)  # Convert type
```

Mini Example Project

```
data = {
    'Name': ['Ali', 'Zara', 'Ahmed', None],
    'Age': [25, 30, None, 28],
    'City': ['Lahore', None, 'Karachi', 'Islamabad'],
    'Salary': [50000, 60000, 55000, None]
}

df = pd.DataFrame(data)

df['Name'].fillna('Unknown', inplace=True)
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['City'].fillna(df['City'].mode()[0], inplace=True)
df['Salary'].fillna(0, inplace=True)

print(df)
```

This fills missing data using different strategies (mean, mode, constant).

Tips for Learning Pandas

- Start with small CSV files to understand `.read_csv()`
 - Use `.head()`, `.info()`, and `.describe()` to explore new data
 - Learn to filter rows and apply custom functions
 - Use `.groupby()` and `.pivot_table()` for summaries
 - Clean data using string and null-handling methods
-

Would you like me to now give **practice questions for Pandas** or move on to **cleaned-up NumPy** with more explanation?

NumPy (Numerical Python) – Detailed Notes with Examples and Explanations

What is NumPy?

NumPy (Numerical Python) is a core scientific computing library in Python. It is used for:

- Working with **arrays and matrices**
- Performing **element-wise operations**
- **Vectorized computations** (faster than loops)
- Useful mathematical functions: **mean, std, dot product, etc.**
- Built-in support for **linear algebra, statistics, random number generation**

It forms the base of many other libraries such as Pandas, SciPy, Scikit-learn, and TensorFlow.

Installation and Import

```
pip install numpy
import numpy as np
```

NumPy Arrays

1. Creating Arrays

A NumPy array is created using `np.array()` from a list or nested list.

```
a = np.array([1, 2, 3])
print(a)
# Output: [1 2 3]
```

2D array:

```
b = np.array([[1, 2], [3, 4]])
print(b)
# Output:
# [[1 2]
#  [3 4]]
```

2. Array Attributes

```
print(b.shape)    # (2, 2) → 2 rows, 2 columns
print(b.ndim)     # 2 → 2 dimensions
print(b.size)     # 4 → total elements
print(b.dtype)    # int64 (platform-dependent)
```

3. Predefined Arrays

These functions quickly create arrays without manually providing data:

```
np.zeros((2, 3))          # 2x3 array filled with 0s
np.ones((3, 3))           # 3x3 array filled with 1s
np.eye(3)                 # 3x3 Identity matrix (diagonal = 1)
np.full((2, 2), 7)        # 2x2 array filled with 7
np.random.rand(2, 2)      # 2x2 array with random floats between 0 and 1
```

4. Array Ranges

Used to create sequences of numbers:

```
np.arange(0, 10, 2)       # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)      # [0. , 0.25, 0.5 , 0.75, 1. ]
```

- `arange()` is like Python's `range` but returns an array.
 - `linspace()` generates evenly spaced numbers between two values.
-

Indexing and Slicing

Same as Python lists but extended to multiple dimensions.

```
a = np.array([10, 20, 30, 40, 50])
print(a[0])          # 10 (first element)
print(a[-1])         # 50 (last element)
print(a[1:4])        # [20 30 40]
```

For 2D arrays:

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b[1, 2])       # 6 (row 1, col 2)
print(b[:, 1])       # [2 5] (all rows, column 1)
```

Reshaping and Flattening

Convert array shapes:

```
a = np.arange(6).reshape(2, 3)
# [[0 1 2]
#   [3 4 5]]
```

```
a_flat = a.flatten()
# [0 1 2 3 4 5]
```

- `reshape()` changes shape without changing data.
 - `flatten()` turns a multi-dimensional array into 1D.
-

Mathematical Operations

All are element-wise (like vectorized operations):

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)      # [5 7 9]
print(a * b)      # [4 10 18]
print(a ** 2)     # [1 4 9]
print(np.sqrt(a)) # [1.0 1.41 1.73]
```

NumPy automatically applies operations element by element.

Aggregate Functions

These summarize arrays:

```
arr = np.array([[1, 2], [3, 4]])

print(np.sum(arr))          # 10 → total of all elements
print(np.sum(arr, axis=0))  # [4 6] → column-wise sum
print(np.mean(arr))        # 2.5 → average
print(np.std(arr))         # 1.118 → standard deviation
print(np.min(arr))         # 1
print(np.max(arr))         # 4
```

Matrix Operations

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[2, 0], [1, 2]])

# Matrix multiplication
print(np.dot(A, B))
# [[ 4  4]
#  [10  8]]

# Transpose of matrix
print(A.T)

# Inverse of matrix
inv = np.linalg.inv(A)
print(inv)

# Determinant
print(np.linalg.det(A))
```

Use `np.dot()` for matrix multiplication, not `*`.

Random Number Generation

```
np.random.seed(42)          # Set seed for reproducibility
np.random.rand(3)           # Random float values [0, 1)
np.random.randint(1, 10, 5) # 5 random integers [1, 9]
np.random.normal(0, 1, (2, 2)) # Normally distributed numbers
```

Broadcasting

Broadcasting automatically adjusts array shapes to perform operations:

```
a = np.array([[1], [2], [3]])      # shape (3,1)
b = np.array([10, 20, 30])        # shape (3,)

print(a + b)
# Output:
# [[11 21 31]
#  [12 22 32]
#  [13 23 33]]
```

Shapes are adjusted: (3,1) and (3,) → (3,3)

Copy vs View (Memory Sharing)

```
a = np.array([1, 2, 3])
b = a          # b is a reference to a
b[0] = 100
print(a)       # [100 2 3]

c = a.copy()   # independent copy
c[0] = 999
print(a)       # [100 2 3]
print(c)       # [999 2 3]
```

Use `copy()` if you don't want the original to change.

Useful Functions and Utilities

```
np.unique([1, 2, 2, 3])          # [1 2 3]
np.sort([3, 1, 2])              # [1 2 3]
np.argsort([3, 1, 2])           # [1 2 0]
np.argmax([10, 50, 30])         # 1 (index of max)
np.where(a > 2, 'Yes', 'No')    # Element-wise condition
```

- `argsort()` returns indices that would sort the array.
 - `where()` returns values based on a condition.
-

Example Mini Project

```
import numpy as np

# Generate random student scores: 5 students, 3 subjects
np.random.seed(1)
scores = np.random.randint(40, 100, size=(5, 3))

print("Scores:\n", scores)

# Mean score per student (row-wise)
```



```
student_avg = np.mean(scores, axis=1)
print("Student Averages:", student_avg)

# Highest scores per subject (column-wise)
highest = np.max(scores, axis=0)
print("Top scores per subject:", highest)
```

Output (Example):

```
Scores:
[[85 47 88]
 [48 59 41]
 [65 75 72]
 [44 43 79]
 [65 87 64]]

Student Averages: [73.3333 49.3333 70.6667 55.3333 72.0000]
Top scores per subject: [85 87 88]
```

Tips for Practicing NumPy

- Start with converting Python lists to arrays and practicing vector math.
- Use `reshape()`, `flatten()`, and broadcasting often.
- Explore `np.linalg` for matrix operations.
- Use random generation to simulate test datasets.
- Combine with Pandas for data cleaning and preprocessing tasks.

Matplotlib – Full Notes with Explanations and Output Examples

1. What is Matplotlib?

Matplotlib is a powerful Python library used to create data visualizations such as line plots, bar charts, histograms, pie charts, scatter plots, and more.

```
import matplotlib.pyplot as plt
```

The main module is `pyplot`, typically imported as `plt`.

2. Basic Line Plot

Purpose: Display trends over a continuous range (like time).

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.plot(x, y)
plt.title("Simple Line Plot")
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.show()
```

Output:

A line graph connecting points:

(1,10), (2,20), (3,15), (4,25), (5,30)

3. Custom Line Styles

Purpose: Improve readability and design of plots.

```
plt.plot(x, y, color='green', linestyle='--', marker='o')
plt.title("Styled Line Plot")
plt.grid(True)
plt.show()
```

Explanation:

- `color='green'` sets the line color.
 - `linestyle='--'` makes it dashed.
 - `marker='o'` adds a circle at each data point.
 - `grid(True)` adds a grid background.
-

4. Bar Chart

Purpose: Compare values across categories.

```
categories = ['Math', 'Physics', 'Chemistry']
scores = [80, 90, 75]
```

```
plt.bar(categories, scores, color='skyblue')
plt.title("Student Scores")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.show()
```

Output:

A vertical bar for each subject showing student marks.

5. Horizontal Bar Chart

```
plt.barh(categories, scores, color='orange')
plt.title("Scores (Horizontal)")
plt.xlabel("Marks")
plt.ylabel("Subjects")
plt.show()
```

Output:

Same as the bar chart but bars are horizontal.

6. Histogram

Purpose: Show the frequency distribution of numeric data.

```
import numpy as np

data = np.random.randn(1000)

plt.hist(data, bins=30, color='purple', edgecolor='black')
plt.title("Histogram of Normally Distributed Data")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Output:

Bars showing how many data points fall into each of 30 equal-width bins.

7. Pie Chart

Purpose: Visualize part-to-whole relationships.

```
sizes = [40, 30, 20, 10]
labels = ['Rent', 'Food', 'Transport', 'Other']
```

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)
plt.title("Monthly Budget Distribution")
```

```
plt.axis('equal')
plt.show()
```

Output:

A pie chart divided into 4 slices with percentage labels.

8. Scatter Plot

Purpose: Display correlation between two numeric variables.

```
x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y, color='red', marker='x')
plt.title("Scatter Plot Example")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```

Output:

Randomly scattered red "x" points on a 2D plot.

9. Subplots

Purpose: Show multiple plots in a single window.

```
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.subplot(1, 2, 1)
plt.plot(x, y1)
plt.title("Sine Function")

plt.subplot(1, 2, 2)
plt.plot(x, y2)
plt.title("Cosine Function")

plt.tight_layout()
plt.show()
```

Output:

Two side-by-side plots: one sine curve, one cosine curve.

10. Line Plot from Pandas DataFrame

```
import pandas as pd

df = pd.DataFrame({
    'Year': [2020, 2021, 2022],
    'Sales': [100, 120, 140]
})

plt.plot(df['Year'], df['Sales'], marker='o')
```

```
plt.title("Yearly Sales")
plt.xlabel("Year")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```

Output:
Line plot showing increase in sales over three years.

11. Save Plot to File

```
plt.plot([1, 2, 3], [3, 2, 1])
plt.title("Test Plot")
plt.savefig("plot.png")      # Save as PNG
plt.savefig("plot.pdf")     # Save as PDF
plt.close()
```

Explanation:
Saves the current figure as an image file. Use `plt.close()` to avoid overwriting when saving multiple plots.

Summary of Plot Types

Function	Plot Type	Use Case
<code>plot()</code>	Line Plot	Show trends or sequences
<code>bar()</code>	Vertical Bar Chart	Compare categories
<code>barh()</code>	Horizontal Bar Chart	Compare categories horizontally
<code>hist()</code>	Histogram	Show distribution of values
<code>scatter()</code>	Scatter Plot	Show correlation
<code>pie()</code>	Pie Chart	Show parts of a whole
<code>subplot()</code>	Multiple in one figure	Compare multiple visuals side-by-side

Would you like a **practice set** or **mini project** using Matplotlib now?

-